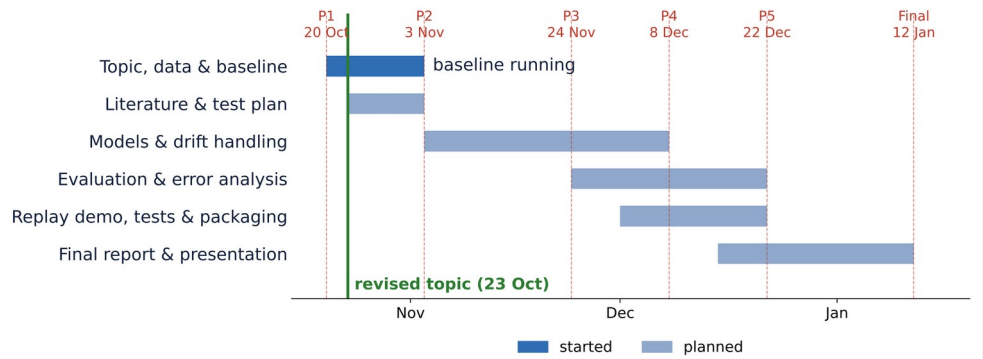
MDM2 – Case Study: Intelligent Systems in Production
One-Page Proposal

Team	Team 1
Members	Yu Han · Melisa Subasi · Meron Hagos · Alejandro Rivas
Roles & Cases	Team lead and core code: Yu Han. Work packages are shared by all four members and tracked on the GitHub issues board.
Project Title	AI-powered Quality Control and Defect Detection in Electronics Assembly: Reducing False Calls of Automated Optical Inspection
GitHub Repository URL	https://github.com/bodhi584/aoi-decision-gate
Contact Email	melisa.subasi@stud.th-deg.de · han.yu@stud.th-deg.de · alejandro.rivas@stud.th-deg.de · meron.hagos@stud.th-deg.de
Industrial Application (target domain/use-case)	Quality inspection in assembly, quality assurance/defect inspection (lecture p. 14): implementation of an AI-powered quality control and defect detection system in electronics assembly, based on the lecture example "Siemens – Quality Control and Defect Detection" (p. 18). Application in manufacturing processes: the step after automated optical inspection (AOI), where flagged boards go to manual inspection. A current industrial problem: on 21 Oct 2026 Siemens and NVIDIA announced an AI second check of AOI results at the Siemens electronics factory in Erlangen (about 40 % fewer false calls targeted). Real data from a Siemens production line in Bavaria.
Keywords (3–6)	AI-powered quality control, defect detection, automated optical inspection, machine learning, neural networks, big data processing
Submission Date (YYYY-MM-DD)	2026-10-23
Gant Chart	
1) Problem Statement & Measurable Outcomes (3–4 sentences)	After soldering, an AI-powered computer vision system (AOI) inspects every electronic component and solder joint and compares each measurement with fixed limits. Flagged boards go to a manual inspection station. In the Siemens data about 98 % of flagged boards are false calls (good boards): a large manual effort, and the line changes over time. Input = AOI measurements of a flagged board; System = AI decision gate (classifier trained on past manual-inspection results, decision-making by threshold); Output = release the board, or send it to manual inspection. Targets, tested in a simulation of the line in time order: at most 1 % of defective boards released, at least 25 % of false-call boards released (baseline: 8.2 %), and an alarm when performance drops.
2) Motivation & Industrial Relevance (2–3 sentences)	Key motivations: reduce dependence on manual quality checks; keep product quality (no more defects released); increase efficiency of the assembly line. Benefits as in the lecture example: improved product quality, reduced waste, increased efficiency. Siemens states that factories in high-wage countries need 7–9 % more productivity every year (Erlangen, Oct 2026).
3) Related Work Snapshot (2–3 key references)	Pfab et al. (2024): Siemens AOI dataset, 132 days, true defect vs false call; notes drift and label errors. Albayari et al. (2025): XGBoost, random forest and neural networks as a second classifier on Siemens AOI data. Schmitt et al. (2020): machine learning at Siemens Amberg predicts inspection results to reduce X-ray inspection. Gap: evaluation in time order as on the real line, a fixed limit on missed defects, and handling of changes over time.
4) Method & Feasibility (≤6 sentences)	Methodology: CRISP-DM. Step 1 Data preparation: Siemens AOI data (Pfab et al. 2024, Mendeley, CC BY-NC 3.0), 440,274 flagged solder joints on 39,742 boards, about 70 AOI measurements plus component type, error code and inspection type, labelled by manual inspection; split strictly by time into train, validate, test. Step 2 Model development: gradient-boosted trees (baseline) and a neural network (PyTorch); threshold set so that at most 1 % of defective boards are released. Step 3 Simulation integration: a day-by-day simulation of the line (train on past days, decide on the next day, retrain on a schedule); boards are virtually routed to release or manual inspection. Step 4 Evaluation and comparison: defective boards released, manual inspections saved, precision, recall; compare models and update policies. Baseline already runs: catching 99 % of defective boards, it releases 8.2 % of false-call boards (29 % with a random split). Deliverables: Python software prototype (command-line tool and web dashboard), tests on GitHub, demo video, report.
5) Milestones & Timeline (short table/list)	P1 23 Oct: proposal + tech stack + MVP (data set, baseline running) P2 3 Nov: core code + baseline sim; test plan + report P3 24 Nov: feature progress (neural network, drift alarm); unit/CI status; issues board P4 8 Dec: feature progress (full evaluation, error analysis); unit/CI status; issues board P5 22 Dec: demo video + run instructions; packaging status Final 12 Jan: final presentation, report (KPIs)
6) Risks & Ethics (1–2 sentences)	Simulation-only environment; no physical machinery risk. Dataset is anonymised (measurements scaled 0–1, no personal data); licence non-commercial (course use only). Ethical AI principles followed: transparency, fairness, accountability; avoid dataset bias (time split, label errors) and ensure reproducibility of results. Main risk: changes over time limit the release rate (fallback: report which handling works and why).